```
// ============================================================
//  SENTINELLE PRO — Robot de trading IA H24 (BTC / ETH / SOL)
//  • Analyse 3 timeframes (15 min / 1 h / 4 h) + régime de marché
//  • IA "gérant de fonds" (Claude) qui valide chaque trade
//  • Sentiment (Fear & Greed) + actualités crypto en direct
//  • Risque pro : 1 %/trade, stop suiveur, prise de gains partielle,
//    pause auto -3 %/jour, coupe-circuit -15 %, pause après 2 pertes
//  • Rapport quotidien 21 h, backtest, discussion avec l'IA
//  MODE DÉMO : 1 000 € virtuels — aucun ordre réel
//  Variables : TELEGRAM_TOKEN, CHAT_ID, ANTHROPIC_API_KEY (optionnel)
// ============================================================
import fs from 'node:fs';

const TOKEN = (process.env.TELEGRAM_TOKEN || '').trim();
const CHAT_ID = (process.env.CHAT_ID || '').trim();
const AI_KEY = (process.env.ANTHROPIC_API_KEY || '').trim();
if (!TOKEN || !CHAT_ID) { console.error('❌ TELEGRAM_TOKEN ou CHAT_ID manquant');

// ---------------- Réglages ----------------
const MARCHES = [
  { nom: 'BTC', pair: 'XBTEUR' },
  { nom: 'ETH', pair: 'ETHEUR' },
  { nom: 'SOL', pair: 'SOLEUR' },
];
const CAPITAL_DEPART = 1000;
const RISQUE = 0.01;            // 1 % du capital risqué par trade
const MAX_PAR_POSITION = 0.4;   // max 40 % du cash par position
const FRAIS = 0.0026;           // frais Kraken par ordre
const PERTE_MAX_JOUR = 0.03;    // -3 % sur la journée → pause jusqu'à demain
const DRAWDOWN_MAX = 0.15;      // -15 % depuis le plus haut → arrêt total
const CONFIANCE_MIN = 65;       // confiance IA minimum
const SCORE_MIN_IA = 70;        // score technique minimum (avec IA)
const SCORE_MIN_SANS_IA = 80;   // plus strict sans IA
const MODELE_DECISION = 'claude-sonnet-5-5';
const MODELE_CHAT = 'claude-haiku-5-5';
const CYCLE_MS = 60_000;
const HEURE_RAPPORT = 21;
const FICHIER = './etat.json';

// ---------------- État ----------------
const etatVide = () => ({
  cash: CAPITAL_DEPART, positions: {}, trades: [], pause: false, pauseAuto: false
  jour: '', valeurDebutJour: CAPITAL_DEPART, plusHautCapital: CAPITAL_DEPART,
```

```
  derniereBougie: {}, dernier: {}, pertesSuite: 0, cooldownJusqua: 0,
  dernierRapport: '', debut: Date.now(),
});
let etat = etatVide();
try { etat = { ...etatVide(), ...JSON.parse(fs.readFileSync(FICHIER, 'utf8')) };
function sauver() {
  fs.writeFileSync(`${FICHIER}.tmp`, JSON.stringify(etat, null, 2));
  fs.renameSync(`${FICHIER}.tmp`, FICHIER);
}

const dormir = (ms) => new Promise((r) => setTimeout(r, ms));
const eur = (n) => `${n.toLocaleString('fr-FR', { minimumFractionDigits: 2, maxim
const pct = (n) => `${n >= 0 ? '+' : ''}${n.toFixed(2)} %`;
const esc = (s) => String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace
const moyenne = (a) => a.reduce((s, x) => s + x, 0) / (a.length || 1);

function paris() {
  const parts = new Intl.DateTimeFormat('fr-FR', {
    timeZone: 'Europe/Paris', year: 'numeric', month: '2-digit', day: '2-digit',
  }).formatToParts(new Date());
  const g = (t) => parts.find((x) => x.type === t).value;
  return { date: `${g('year')}-${g('month')}-${g('day')}`, heure: Number(g('hour'
}

const cache = {};
async function enCache(cle, ttl, fn) {
  const c = cache[cle];
  if (c && Date.now() - c.t < ttl) return c.v;
  const v = await fn();
  cache[cle] = { t: Date.now(), v };
  return v;
}

// ---------------- Telegram ----------------
async function tg(methode, corps) {
  const r = await fetch(`https://api.telegram.org/bot${TOKEN}/${methode}`, {
    method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.s
  });
  return r.json();
}
async function envoyer(texte) {
  try {
    await tg('sendMessage', { chat_id: CHAT_ID, text: texte.slice(0, 4000), parse
  } catch (e) { console.log('Telegram :', e.message); }
}
```

```
// ---------------- Données marché ----------------
async function kraken(chemin) {
  const r = await fetch(`https://api.kraken.com/0/public/${chemin}`);
  const j = await r.json();
  if (j.error?.length) throw new Error(j.error.join(', '));
  return j.result;
}
function ohlc(pair, intervalle) {
  const ttl = { 15: 55_000, 60: 300_000, 240: 900_000 }[intervalle];
  return enCache(`ohlc-${pair}-${intervalle}`, ttl, async () => {
    const res = await kraken(`OHLC?pair=${pair}&interval=${intervalle}`);
    const cle = Object.keys(res).find((k) => k !== 'last');
    return res[cle].map((c) => ({ t: c[0], h: +c[2], l: +c[3], c: +c[4], v: +c[6]
  });
}
async function prix(pair) {
  const res = await kraken(`Ticker?pair=${pair}`);
  return +res[Object.keys(res)[0]].c[0];
}
function fearGreed() {
  return enCache('fng', 3_600_000, async () => {
    try {
      const j = await (await fetch('https://api.alternative.me/fng/?limit=1')).js
      return { valeur: +j.data[0].value, label: j.data[0].value_classification };
    } catch { return null; }
  });
}
function actus() {
  return enCache('actus', 1_800_000, async () => {
    try {
      const x = await (await fetch('https://www.coindesk.com/arc/outboundfeeds/rs
      return [...x.matchAll(/<title>(?:<!\[CDATA\[)?([\s\S]*?)(?:\]\]>)?<\/title>
    } catch { return []; }
  });
}

// ---------------- Indicateurs ----------------
function ema(v, n) {
  const k = 2 / (n + 1); let e = v[0]; const out = [e];
  for (let i = 1; i < v.length; i++) { e = v[i] * k + e * (1 - k); out.push(e); }
  return out;
}
function rma(v, n) {
  const out = []; let s = 0;
  for (let i = 0; i < v.length; i++) {
    if (i < n) { s += v[i]; out.push(s / (i + 1)); } else out.push((out[i - 1] *
```

```
  }
  return out;
}
function rsi(v, n = 14) {
  const g = [0]; const p = [0];
  for (let i = 1; i < v.length; i++) { const d = v[i] - v[i - 1]; g.push(Math.max
  const ag = rma(g, n); const ap = rma(p, n);
  return ag.map((x, i) => (ap[i] === 0 ? 100 : 100 - 100 / (1 + x / ap[i])));
}
function adxAtr(b, n = 14) {
  const pdm = [0]; const ndm = [0]; const tr = [b[0].h - b[0].l];
  for (let i = 1; i < b.length; i++) {
    const haut = b[i].h - b[i - 1].h; const bas = b[i - 1].l - b[i].l;
    pdm.push(haut > bas && haut > 0 ? haut : 0);
    ndm.push(bas > haut && bas > 0 ? bas : 0);
    tr.push(Math.max(b[i].h - b[i].l, Math.abs(b[i].h - b[i - 1].c), Math.abs(b[i
  }
  const atr = rma(tr, n); const sp = rma(pdm, n); const sn = rma(ndm, n);
  const dx = atr.map((a, i) => {
    const pdi = a ? (100 * sp[i]) / a : 0; const ndi = a ? (100 * sn[i]) / a : 0;
    return pdi + ndi ? (100 * Math.abs(pdi - ndi)) / (pdi + ndi) : 0;
  });
  return { adx: rma(dx, n), atr };
}
function indic(b) {
  const c = b.map((x) => x.c); const v = b.map((x) => x.v);
  const e20 = ema(c, 20); const e50 = ema(c, 50); const e200 = ema(c, 200);
  const r = rsi(c); const { adx, atr } = adxAtr(b);
  const i = c.length - 1;
  const d20 = c.slice(-20); const m20 = moyenne(d20);
  const sd = Math.sqrt(moyenne(d20.map((x) => (x - m20) ** 2)));
  return {
    t: b[i].t, prix: c[i], e20: e20[i], e50: e50[i], e200: e200[i],
    rsi: r[i], rsiAvant: r[i - 1], adx: adx[i], atr: atr[i], atrMoy: moyenne(atr.
    bbHaut: m20 + 2 * sd, bbBas: m20 - 2 * sd, bbMilieu: m20,
    vol: v[i], volMoy: moyenne(v.slice(-21, -1)),
    haussier: e20[i] > e50[i] && c[i] > e50[i],
    baissier: e20[i] < e50[i] && c[i] < e50[i],
  };
}

// ---------------- Cerveau : analyse complète d'un marché ----------------
async function analyseMarche(m) {
  const [b15, b1h, b4h] = await Promise.all([ohlc(m.pair, 15), ohlc(m.pair, 60),
  const t15 = indic(b15); const t1h = indic(b1h); const t4h = indic(b4h);
  const c1h = b1h.map((x) => x.c);
```

```
  const var24h = (c1h.at(-1) / c1h.at(-25) - 1) * 100;
  const fng = await fearGreed();

  let regime = 'NEUTRE';
  if (var24h < -7 || t1h.atr > 2 * t1h.atrMoy) regime = 'CHAOS';
  else if (t1h.adx >= 22 && (t1h.haussier || t1h.baissier)) regime = 'TENDANCE';
  else if (t1h.adx < 18) regime = 'RANGE';

  let signal = null;
  // Stratégie 1 — suivi de tendance : 4 h + 1 h haussiers, rebond du RSI 15 min
  if (regime === 'TENDANCE' && t4h.haussier && t1h.haussier
      && t15.rsiAvant < 50 && t15.rsi >= 50 && t15.rsi < 70 && t15.prix > t15.e50
    let score = 45;
    if (t4h.prix > t4h.e200) score += 15;
    if (t1h.adx > 28) score += 10;
    if (t15.vol > t15.volMoy * 1.2) score += 10;
    if (!fng || fng.valeur < 80) score += 10;
    if (t1h.rsi < 68) score += 10;
    signal = { strat: 'TENDANCE', score, R: 1.5 * t1h.atr };
  // Stratégie 2 — retour à la moyenne en marché calme : excès de vente sur 15 mi
  } else if (regime === 'RANGE' && !t4h.baissier && t15.prix < t15.bbBas && t15.r
    let score = 50;
    if (t15.vol > t15.volMoy * 1.5) score += 10;
    if (fng && fng.valeur < 35) score += 15;
    if (t1h.rsi < 40) score += 15;
    if (t4h.prix > t4h.e200) score += 10;
    signal = { strat: 'RANGE', score, R: 1.2 * t1h.atr, tp: t15.bbMilieu };
  }
  return { m, t15, t1h, t4h, var24h, regime, signal, fng };
}

// ---------------- IA ----------------
const SYSTEME = `Tu es le gérant principal d'un fonds quantitatif crypto. Tu as t
Ta règle d'or : protéger le capital d'abord. Tu ne prends que les trades à forte
Tu raisonnes sur : l'alignement des timeframes, le régime de marché, la volatilit

async function claude(modele, systeme, contenu, maxTokens) {
  const r = await fetch('https://api.anthropic.com/v1/messages', {
    method: 'POST',
    headers: { 'x-api-key': AI_KEY, 'anthropic-version': '2023-06-01', 'content-t
    body: JSON.stringify({ model: modele, max_tokens: maxTokens, system: systeme,
  });
  const j = await r.json();
  if (!j.content?.[0]?.text) throw new Error(j.error?.message || 'réponse vide');
  return j.content[0].text;
}
```

```
function statsStrategie() {
  const derniers = etat.trades.slice(-20);
  return {
    trades: derniers.length,
    gagnants: derniers.filter((x) => x.gain > 0).length,
    resultat: Math.round(derniers.reduce((s, x) => s + x.gain, 0) * 100) / 100,
  };
}

async function avisIA(A) {
  if (!AI_KEY) return { decision: 'ACHETER', confiance: null, risque: 0.7, raison
  const r = (x) => Math.round(x * 100) / 100;
  const tf = (t) => ({ prix: r(t.prix), ema20: r(t.e20), ema50: r(t.e50), ema200:
  const donnees = {
    actif: `${A.m.nom}/EUR`, strategie: A.signal.strat, score_technique: A.signal
    regime: A.regime, variation_24h_pct: r(A.var24h),
    tf_15min: tf(A.t15), tf_1h: tf(A.t1h), tf_4h: tf(A.t4h),
    stop_loss_eur: r(A.t15.prix - A.signal.R), risque_par_unite_eur: r(A.signal.R
    fear_greed: A.fng, actualites: await actus(), resultats_recents: statsStrateg
  };
  const consigne = `Un signal d'achat vient de sortir. Données :\n${JSON.stringif
    + 'Décide en gérant de fonds. Réponds UNIQUEMENT avec ce JSON :\n'
    + '{"decision":"ACHETER" ou "ATTENDRE","confiance":0-100,"risque":0.3-1 (tail
  try {
    const texte = await claude(MODELE_DECISION, SYSTEME, consigne, 300);
    const m = texte.match(/\{[\s\S]*\}/);
    if (!m) throw new Error('format');
    return JSON.parse(m[0]);
  } catch (e) {
    return { decision: 'ATTENDRE', confiance: 0, risque: 0, raison: `IA injoignab
  }
}

// ---------------- Portefeuille virtuel ----------------
async function valeurTotale() {
  let v = etat.cash;
  for (const [nom, p] of Object.entries(etat.positions)) {
    v += p.qty * (await prix(MARCHES.find((x) => x.nom === nom).pair));
  }
  return v;
}

async function ouvrir(A, ia) {
  const { m, t15, signal } = A;
  const p = t15.prix; const R = signal.R;
```

```
  const capital = await valeurTotale();
  const mult = Math.min(1, Math.max(0.3, Number(ia.risque) || 0.7));
  let qty = (capital * RISQUE * mult) / R;
  let cout = qty * p;
  if (cout > etat.cash * MAX_PAR_POSITION) { cout = etat.cash * MAX_PAR_POSITION;
  if (cout < 10) return;
  const frais = cout * FRAIS;
  etat.cash -= cout + frais;
  etat.positions[m.nom] = {
    strat: signal.strat, qty, entree: p, sl: p - R, R, tp: signal.tp ?? p + 2 * R
    plusHaut: p, atr: A.t1h.atr, securise: false, partiel: false,
    cout, frais, encaisse: 0, date: Date.now(),
  };
  sauver();
  await envoyer(
    `🟢 <b>ACHAT ${m.nom}</b> · ${signal.strat === 'TENDANCE' ? 'suivi de tendanc
    + `Prix : ${eur(p)} · Montant : ${eur(cout)}\n`
    + `🛑 Stop : ${eur(p - R)} · 🎯 Objectif : ${eur(signal.tp ?? p + 2 * R)}\n`
    + `📊 Score technique : ${signal.score}/100 · Régime : ${A.regime}\n`
    + `🤖 IA : ${ia.confiance === null ? 'non activée' : `${ia.confiance} % de cc
  );
}

function vendre(nom, qty, prixVente) {
  const pos = etat.positions[nom];
  const recu = qty * prixVente; const frais = recu * FRAIS;
  etat.cash += recu - frais;
  pos.encaisse += recu - frais;
  pos.frais += frais;
  pos.qty -= qty;
}

async function fermer(nom, prixSortie, motif) {
  const pos = etat.positions[nom];
  vendre(nom, pos.qty, prixSortie);
  // encaissé (net des frais de vente) − coût d'achat − frais d'achat
  const gainNet = pos.encaisse - pos.cout - pos.cout * FRAIS;
  const gainPct = (gainNet / pos.cout) * 100;
  etat.trades.push({ nom, strat: pos.strat, entree: pos.entree, sortie: prixSorti
  delete etat.positions[nom];
  if (gainNet < 0) {
    etat.pertesSuite += 1;
    if (etat.pertesSuite >= 2) {
      etat.cooldownJusqua = Date.now() + 2 * 3_600_000;
      etat.pertesSuite = 0;
      await envoyer('🧊 2 pertes d\'affilée : Sentinelle se calme 2 h avant de re
```

```
    }
  } else etat.pertesSuite = 0;
  sauver();
  await envoyer(
    `${gainNet >= 0 ? '✅' : '🔴'} <b>VENTE ${nom}</b> (démo) · ${motif}\n`
    + `Entrée ${eur(pos.entree)} → Sortie ${eur(prixSortie)}\n`
    + `Résultat : <b>${gainNet >= 0 ? '+' : ''}${eur(gainNet)}</b> (${pct(gainPct
  );
}

async function gerer(nom, p) {
  const pos = etat.positions[nom];
  pos.plusHaut = Math.max(pos.plusHaut, p);

  if (p <= pos.sl) return fermer(nom, p, pos.securise ? 'stop suiveur, gains sécu

  if (pos.strat === 'RANGE') {
    if (p >= pos.tp) return fermer(nom, p, 'retour à la moyenne 🎯');
    if (Date.now() - pos.date > 12 * 3_600_000) return fermer(nom, p, 'trade trop
    return;
  }
  // Tendance : sécuriser à +1R, encaisser la moitié à +2R, puis laisser courir a
  if (!pos.securise && p >= pos.entree + pos.R) {
    pos.sl = Math.max(pos.sl, pos.entree * (1 + 2 * FRAIS));
    pos.securise = true; sauver();
    await envoyer(`🔒 ${nom} : stop remonté au prix d'entrée, ce trade ne peut pl
  }
  if (!pos.partiel && p >= pos.entree + 2 * pos.R) {
    const avant = pos.encaisse;
    vendre(nom, pos.qty / 2, p);
    pos.partiel = true; sauver();
    await envoyer(`💰 ${nom} : moitié vendue à ${eur(p)} (+${eur(pos.encaisse - a
  }
  if (pos.partiel) {
    const suiveur = pos.plusHaut - 2 * pos.atr;
    if (suiveur > pos.sl) { pos.sl = suiveur; sauver(); }
  }
}

// ---------------- Boucle H24 ----------------
async function gererJour() {
  const { date, heure } = paris();
  if (etat.jour !== date) {
    etat.jour = date;
    etat.valeurDebutJour = await valeurTotale();
    if (etat.pauseAuto) { etat.pause = false; etat.pauseAuto = false; await envoy
```

```
    sauver();
  }
  if (heure >= HEURE_RAPPORT && etat.dernierRapport !== date) {
    etat.dernierRapport = date; sauver();
    await rapport();
  }
}

async function controleRisque() {
  const v = await valeurTotale();
  etat.plusHautCapital = Math.max(etat.plusHautCapital, v);
  if (!etat.pause && v < etat.valeurDebutJour * (1 - PERTE_MAX_JOUR)) {
    etat.pause = true; etat.pauseAuto = true;
    await envoyer(`⛔ Perte max du jour atteinte (${pct((v / etat.valeurDebutJour
  }
  if (!etat.pause && v < etat.plusHautCapital * (1 - DRAWDOWN_MAX)) {
    etat.pause = true; etat.pauseAuto = false;
    await envoyer(`🚨 <b>Coupe-circuit</b> : -${Math.round(DRAWDOWN_MAX * 100)} %
  }
  sauver();
}

async function cycle() {
  await gererJour();
  for (const m of MARCHES) {
    if (etat.positions[m.nom]) await gerer(m.nom, await prix(m.pair));
  }
  for (const m of MARCHES) {
    const A = await analyseMarche(m);
    etat.dernier[m.nom] = { prix: A.t15.prix, regime: A.regime, rsi15: Math.round
    if (etat.derniereBougie[m.nom] === A.t15.t) continue; // attendre une nouvell
    etat.derniereBougie[m.nom] = A.t15.t;
    sauver();

    const pos = etat.positions[m.nom];
    if (pos?.strat === 'TENDANCE' && A.t1h.baissier) { await fermer(m.nom, A.t15.
    if (pos || !A.signal || etat.pause || Date.now() < etat.cooldownJusqua) conti
    if (A.signal.score < (AI_KEY ? SCORE_MIN_IA : SCORE_MIN_SANS_IA)) continue;

    const ia = await avisIA(A);
    if (ia.decision === 'ACHETER' && (ia.confiance === null || ia.confiance >= CO
    else await envoyer(`⚪ Signal ${m.nom} (score ${A.signal.score}) refusé par l
  }
  await controleRisque();
}
```

```
// ---------------- Rapport & backtest ----------------
async function rapport() {
  const v = await valeurTotale();
  const aujourdhui = etat.trades.filter((x) => Date.now() - x.date < 86_400_000);
  const gagnants = etat.trades.filter((x) => x.gain > 0).length;
  const pire = etat.trades.reduce((min, x) => Math.min(min, x.gain), 0);
  const jours = Math.max(1, Math.round((Date.now() - etat.debut) / 86_400_000));
  let marche = '';
  for (const [nom, d] of Object.entries(etat.dernier)) marche += `\n${nom} ${eur(
  await envoyer(
    `📊 <b>Rapport Sentinelle</b> (jour ${jours})\n`
    + `Capital : <b>${eur(v)}</b> · ${pct((v / CAPITAL_DEPART - 1) * 100)} depuis
    + `Aujourd'hui : ${pct((v / etat.valeurDebutJour - 1) * 100)} · ${aujourdhui.
    + `Réussite totale : ${etat.trades.length ? Math.round((gagnants / etat.trade
    + `Pire trade : ${eur(pire)}\n`
    + `Plus haut atteint : ${eur(etat.plusHautCapital)}\n${marche}`,
  );
}

function simuler(b) {
  const c = b.map((x) => x.c);
  const e20 = ema(c, 20); const e50 = ema(c, 50); const e200 = ema(c, 200);
  const r = rsi(c); const { atr } = adxAtr(b);
  let cap = 1000; let pic = 1000; let dd = 0; let pos = null; const res = [];
  const sortir = (q, px) => { const pnl = q * (px - pos.e) - q * (px + pos.e) * F
  for (let i = 200; i < b.length; i++) {
    const x = b[i];
    if (pos) {
      pos.haut = Math.max(pos.haut, x.h);
      if (x.l <= pos.sl) { sortir(pos.q, pos.sl); res.push(pos.res); pos = null;
        if (!pos.sec && x.h >= pos.e + pos.R) { pos.sl = pos.e; pos.sec = true; }
        if (!pos.part && x.h >= pos.e + 2 * pos.R) { sortir(pos.q / 2, pos.e + 2
        if (pos.part) pos.sl = Math.max(pos.sl, pos.haut - 2 * pos.atr);
        if (e20[i] < e50[i]) { sortir(pos.q, x.c); res.push(pos.res); pos = null;
      }
    } else if (e20[i] > e50[i] && x.c > e200[i] && r[i - 1] < 50 && r[i] >= 50 &&
      const R = 1.5 * atr[i];
      const q = Math.min((cap * RISQUE) / R, (cap * MAX_PAR_POSITION) / x.c);
      pos = { e: x.c, sl: x.c - R, R, q, haut: x.c, sec: false, part: false, atr:
    }
    pic = Math.max(pic, cap); dd = Math.max(dd, (1 - cap / pic) * 100);
  }
  return { cap, n: res.length, g: res.filter((x) => x > 0).length, dd, jours: Mat
}

async function backtest() {
```

```
  let t = '🧪 <b>Backtest</b> (règles de tendance, bougies 1 h, sans IA)\n';
  for (const m of MARCHES) {
    const s = simuler(await ohlc(m.pair, 60));
    t += `\n<b>${m.nom}</b> sur ${s.jours} jours : ${eur(s.cap)} (${pct((s.cap /
      + `${s.n} trades · ${s.n ? Math.round((s.g / s.n) * 100) : 0} % gagnants ·
  }
  t += '\n<i>Le passé ne garantit pas le futur : c\'est la démo en direct qui com
  await envoyer(t);
}

// ---------------- Commandes Telegram ----------------
async function commande(texte) {
  const cmd = texte.split(/[\s@]/)[0].toLowerCase();
  if (!cmd.startsWith('/')) return discuter(texte);

  if (cmd === '/start' || cmd === '/aide') {
    return envoyer(
      '🛡️ <b>Sentinelle Pro</b> — trading IA H24 (démo)\n\n'
      + '/status — état du robot\n/portefeuille — positions en cours\n/historique
      + '/analyse — analyse complète maintenant\n/rapport — rapport du jour\n/bac
      + '/pause · /reprendre · /reset\n\n💬 Écris-moi n\'importe quelle question,
    );
  }
  if (cmd === '/status') {
    const v = await valeurTotale();
    const cool = Date.now() < etat.cooldownJusqua ? `\n🆘 Pause anti-tilt jusqu'à
    return envoyer(
      `🛡️ <b>Sentinelle Pro</b> ${etat.pause ? '⏸️ en pause' : '▶️ actif H24'}\n
      + `Capital : <b>${eur(v)}</b> (${pct((v / CAPITAL_DEPART - 1) * 100)})\n`
      + `Positions : ${Object.keys(etat.positions).length} · Trades : ${etat.trad
      + `🤖 IA : ${AI_KEY ? 'activée' : 'pas encore activée'}${cool}`,
    );
  }
  if (cmd === '/portefeuille') {
    let t = `💼 <b>Portefeuille démo</b>\nCash : ${eur(etat.cash)}\n`;
    const liste = Object.entries(etat.positions);
    if (!liste.length) t += '\nAucune position ouverte, Sentinelle attend une vra
    for (const [nom, p] of liste) {
      const actuel = await prix(MARCHES.find((x) => x.nom === nom).pair);
      t += `\n<b>${nom}</b> (${p.strat}) : ${eur(p.entree)} → ${eur(actuel)} (${p
        + `🛑 ${eur(p.sl)}${p.securise ? ' 🔒' : ''} · 🎯 ${eur(p.tp)}${p.partie
    }
    return envoyer(t);
  }
  if (cmd === '/historique') {
    if (!etat.trades.length) return envoyer('Aucun trade pour l\'instant. Un pro
```

```
    const g = etat.trades.filter((x) => x.gain > 0).length;
    const total = etat.trades.reduce((s, x) => s + x.gain, 0);
    const lignes = etat.trades.slice(-10).reverse()
      .map((x) => `${x.gain >= 0 ? '✅' : '🔴'} ${x.nom} ${pct(x.gainPct)} (${eur
    return envoyer(`📜 <b>Historique</b>\nRéussite : ${Math.round((g / etat.trade
  }
  if (cmd === '/analyse') {
    let t = '🔎 <b>Analyse en direct</b>\n';
    const fng = await fearGreed();
    if (fng) t += `Sentiment : ${fng.valeur}/100 (${fng.label})\n`;
    for (const m of MARCHES) {
      const A = await analyseMarche(m);
      const tend = (x) => (x.haussier ? '📈' : x.baissier ? '📉' : '➖');
      t += `\n<b>${m.nom}</b> ${eur(A.t15.prix)} · 24 h ${pct(A.var24h)}\n`
        + `Régime : ${A.regime} · 4h ${tend(A.t4h)} 1h ${tend(A.t1h)} 15m ${tend(
        + `Signal : ${A.signal ? `${A.signal.strat} (score ${A.signal.score})` :
    }
    return envoyer(t);
  }
  if (cmd === '/rapport') return rapport();
  if (cmd === '/backtest') { await envoyer('🧪 Calcul en cours…'); return backtes
  if (cmd === '/pause') { etat.pause = true; etat.pauseAuto = false; sauver(); re
  if (cmd === '/reprendre') { etat.pause = false; etat.pauseAuto = false; etat.co
  if (cmd === '/reset') { etat = etatVide(); sauver(); return envoyer('🔄 Démo re
  return envoyer('Commande inconnue. Envoie /aide');
}

async function discuter(texte) {
  if (!AI_KEY) return envoyer('🤖 L\'IA n\'est pas encore activée. Ajoute ta clé
  try {
    const contexte = { marches: etat.dernier, positions: etat.positions, capital:
    const rep = await claude(
      MODELE_CHAT,
      `${SYSTEME}\nTu parles à ton client en français, simplement et franchement,
      `Contexte du robot : ${JSON.stringify(contexte)}\n\nQuestion : ${texte}`,
      500,
    );
    return envoyer(esc(rep));
  } catch (e) { return envoyer(`IA indisponible : ${esc(e.message)}`); }
}

let offset = 0;
async function ecouter() {
  while (true) {
    try {
      const j = await tg('getUpdates', { offset, timeout: 25 });
```

```
      for (const u of j.result || []) {
        offset = u.update_id + 1;
        const msg = u.message;
        if (!msg?.text || String(msg.chat.id) !== CHAT_ID) continue; // n'obéit q
        await commande(msg.text.trim());
      }
    } catch (e) { console.log('Écoute :', e.message); await dormir(5000); }
  }
}

// ---------------- Démarrage ----------------
process.on('unhandledRejection', (e) => console.log('Erreur :', e?.message || e))
await envoyer(`✅ <b>Sentinelle Pro est en ligne</b>\nH24 · BTC, ETH, SOL · démo
console.log('Sentinelle Pro démarrée');
ecouter();
while (true) {
  try { await cycle(); } catch (e) { console.log('Cycle :', e.message); }
  await dormir(CYCLE_MS);
}
```